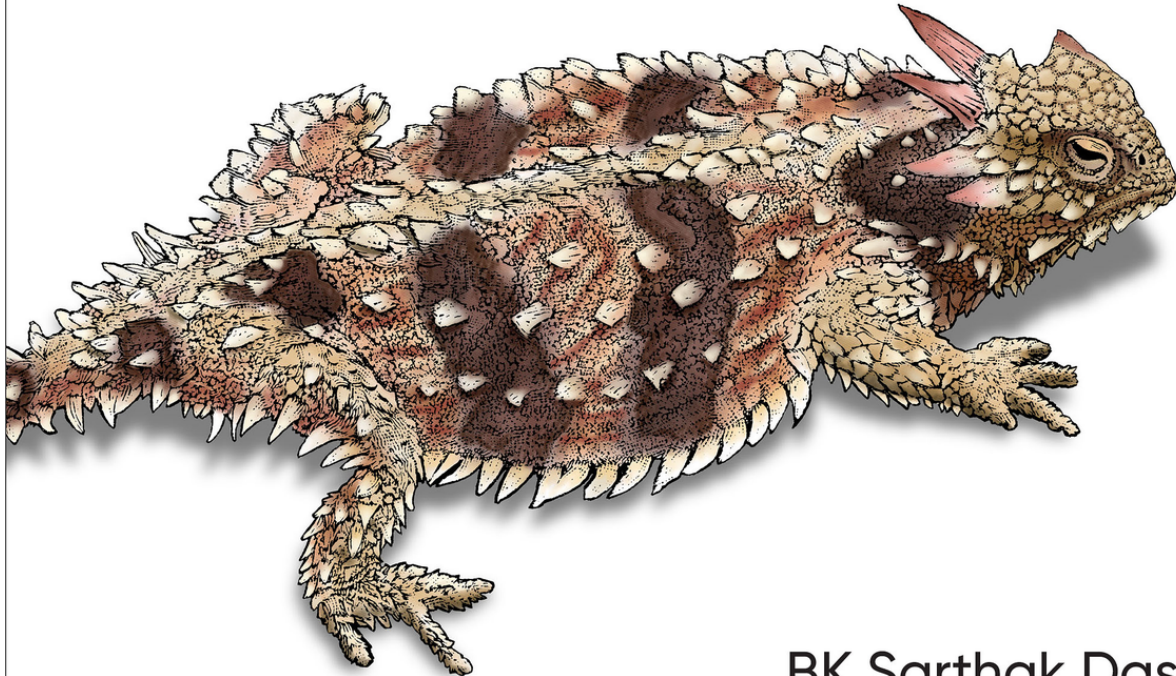


O'REILLY®

Security as Code

DevSecOps Patterns with AWS



BK Sarthak Das
& Virginia Chu

Security as Code

DevSecOps Patterns with AWS

BK Sarthak Das and Virginia Chu

Security as Code

by BK Sarthak Das and Virginia Chu

Copyright © 2023 Virginia Chu and BK Sarthak Das. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

- Acquisitions Editors: Jennifer Pollock, Simina Calin
- Development Editor: Sarah Grey
- Production Editor: Clare Laylock
- Copyeditor: Nicole Taché
- Proofreader: Rachel Head
- Indexer: Sam Arnold-Boyd
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery
- Illustrator: Kate Dullea
- January 2023: First Edition

Revision History for the First Edition

- 2023-01-03: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098127466> for release details.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Security as Code*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O'Reilly and NGINX. See our [statement of editorial independence](#).

978-1-098-12746-6

[LSI]

Preface

The authors of this book work with enterprise AWS customers who have business-critical applications running in the cloud, so we think about security on a daily basis. In recent years, we've noticed that the term *DevSecOps* pops up in nearly every security strategy discussion. Everyone wants it, but not as many people understand it—and it seems like almost nobody knows where to start or what to do.

DevSecOps is a relatively new field, and few books are available to guide those who want to learn more about it. We decided to write this book to help fill that gap by showing you how and where to get started on DevSecOps in AWS.

This book is not an enterprise-grade solution kit for copying and pasting into production (and since every project and organization has different needs, we sincerely hope you would never do that!). Instead, it's designed to introduce you to the building blocks of the DevSecOps mindset, and to guide you along the way with practical examples. We use popular open source tools where possible, to show you that it's not always necessary to buy expensive products to do security the right way.

We use a fictitious company called Automatoonz to illustrate some of the real-world issues you're likely to face in your DevSecOps journey. As we discuss a problem, the Automatoonz team works on it too, giving you a sense of how real teams approach solving the problem at hand. Although the scenarios are fictionalized, these examples come from our extensive personal experience, and we think they'll resonate with you. The solutions we provide in this book are intended as guidance on the art of the possible.

Who Is This Book For?

This book is for AWS security engineers, DevOps engineers, security analysts, security engineering managers, and other practitioners and leaders at intermediate and senior levels who want to automate more of their security. We recommend that readers have some practical AWS development knowledge and familiarity with Git before starting this book: ideally, enough to do basic coding and debugging within AWS. In **Chapter 2**, for example, we use CloudFormation, Python, and Kubernetes to demonstrate Infrastructure as Code. You should also be comfortable navigating Git repositories.

What Do You Need To Get Started?

In practical terms, aside from intermediate knowledge of AWS, to follow the exercises in this book you will need an AWS account where you can deploy. You will also need to install the following, if you do not already have them:

- AWS Command Line Interface (AWS CLI) (latest version)
- Access to an AWS account
- Docker (Community Edition)
- Python (version 3.x.x or higher)
- Git (latest version)
- Kubectl (latest version)
- Kubernetes (version 1.21 or higher)

Chapter 2 has a detailed walkthrough of setting up all these tools.

You will also need access to the book's **GitHub repository**, which includes code samples and other supplemental materials.

What's in This Book?

We've tried to ensure that the seven chapters in this book are as independent as possible from one another, so that you can pick it up at any point. However, we recommend that you start from the beginning.

Chapter 1 will introduce you to what DevSecOps is, why it is important, and what kind of mindset you'll need to get started. **Chapter 2** helps you install the software you'll need for the rest of the book, then walks you through a sample application built with secure configurations to ensure you have your toolkit working. In **Chapter 3**, you'll learn how to validate Infrastructure as Code to make your

resources secure. **Chapter 4** looks at how to set up appropriate logging and monitoring to identify and debug issues with your infrastructure.

In **Chapter 5**, you'll learn about controlling access through automation, including assessing your organization's identity and access management (IAM) policies and refining them according to the principle of least privilege. **Chapter 6** is all about testing: we'll introduce you to the practice of Chaos Engineering, show you how to use it to make your infrastructure more resilient, and discuss how to focus on possible points of failure. Finally, in **Chapter 7**, we wrap up with a look at the roles and processes that should be part of any DevSecOps team.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

`Constant width italic`

Shows text that should be replaced with user-supplied values or by values determined by context.

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

WARNING

This element indicates a warning or caution.

Using Code Examples

Code examples are available for download at <https://oreil.ly/SaCgh>.

If you have a technical question or a problem using the code examples, please send email to bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Security as Code: DevSecOps Patterns with AWS* by BK Sarthak Das and Virginia Chu (O'Reilly). Copyright 2023 Virginia Chu and BK Sarthak Das, 978-1-098-12746-6.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Sebastopol, CA 95472

800-998-9938 (in the United States or Canada)

707-829-0515 (international or local)

707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/SecurityAsCode>.

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>.

Follow us on Twitter: <https://twitter.com/oreillymedia>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

There are numerous people who are responsible for this book.

We would like to take the time to thank our amazing and wonderful team at O'Reilly Media (Sarah Grey, development editor; Nicole Taché, freelance editor; and Clare Laylock, production editor) for their professionalism, commitment, and guidance in publishing a book. We learned that writing a book is not easy or trivial.

Our grateful and humble thanks go to our technical reviewer Joe Milligan. It has been an honor to collaborate with him.

BK would like to thank his close friends and family for being supportive throughout the process of writing this book.

Virginia would like to thank her family, four-legged sounding board, and close friends for their support and patience throughout this adventure. Special thanks to her mentor Michael Hausenblas for inspiring her to work on this book.

Finally, we would like to thank all the wonderful contributors and developers for the tools we have used in our book.

Chapter 1. Introduction to DevSecOps

We did not start our careers in security, but we both know that delivering software is of utmost importance to developers. “Delivering software” in this context means delivering *something that does what it is supposed to do*. This could refer to the code being stable, or the software meeting the functional requirements (for example, a calculator can add, multiply, subtract, and divide numbers) and performance expectations without any issues (for example, a chat application allows more than 10 people to send messages to each other simultaneously).

Building *quality* software, however, requires good coding practices, resilient architectures, and security. It’s common for security to be added into the software after it has been built, but the *shift-left approach* recommends moving security to much earlier in the development life cycle, building it in from the start. We will discuss that approach in this chapter, focusing on *Security as Code* (SaC). In this approach, our infrastructure’s security policies—detection, prevention, remediation—are all defined as code.

We will also discuss DevSecOps in this chapter, focusing on the three major players in software development:

Development

Is your code doing what it is meant to do?

Operations

What is this code running on? Do you have the required skills/time to maintain this going forward?

Can the provisioned infrastructure handle the expected workload?

Testing

Can the code survive unexpected use cases? How does the code respond to something you didn’t account for?

The primary focus of this book is how to integrate security into your development process through cloud infrastructure. Declaring infrastructure in files is also known as *Infrastructure as Code* (IaC). Kief Morris provides a helpful definition in his book *Infrastructure as Code, 2nd edition* (O’Reilly):

Infrastructure as Code is an approach to infrastructure automation based on practices from software development. It emphasizes consistent, repeatable routines for provisioning and changing systems and their configuration. You make changes to code, then use automation to test and apply those changes to your systems.

Using IaC and declaring the security aspects of that infrastructure is Security as Code (SaC). SaC is not entirely different from IaC, but rather focuses on enabling security controls using templates.

This chapter introduces the basic concepts of SaC, and **Chapter 2** provides setup and instructions to get you started. After that, the chapters are organized by security domains. Some chapters are exclusive to a single domain; others address multiple domains in a single stage of the buildout. Each domain has its own unique set of questions. For instance:

Data protection (Chapter 3)

Is everything encrypted? Does the encryption approach follow data classification policies? Do we have data classification implemented?

Infrastructure security (Chapter 3)

As we are running our application in the cloud, is all the infrastructure deployed securely? Is our S3 bucket publicly accessible when it should not be? Can we prevent deployment of our resources when something is missing?

Application security (Chapter 3)

Is the code we're running secure? How many active vulnerabilities are there in our code? Did we release code with a critical vulnerability?

Logging and monitoring (Chapter 4)

Do we know what to monitor? When does an observation become an anomaly? Are the right application security indicators in place to inform us of any mishap?

Identity and access management (Chapter 5)

Who has access to what? Does anyone have any elevated privileges? Can someone elevate their own or others' privileges?

Incident response (Chapter 6)

How do we react to incidents? Can we replace part of the offending application when something goes wrong?

Throughout the chapters, we will use Amazon Web Services (AWS) native security services and best practices to baseline the environment we are deploying. When we assume certain operational and team structures, we explicitly call out those assumptions.

Before DevOps: The Software Development Life Cycle

When you are new in your career, writing a piece of code that does exactly what was asked is a joy of its own. With time, however, we've realized writing quality code does not stop at making $2 + 2 = 4$. What if the user enters " $2 + a$ "? How should your software behave?

Well, that's the responsibility of the Quality and Testing engineers, isn't it? Wrong.

We've seen back-and-forth between developers and testers that was time-consuming and created unhealthy expectations from both teams. Let's take the example of the calculator input of " $2 + a$ ". If, as a coder, you did not think of this use case because it was not part of the requirements, and your tester or QA team did not record it as a test case, you would be shipping broken code to your customers. This broken code would be your final product. A codebase that doesn't work as expected is not a joy for the end user to work with.

Code needs to be hosted on some infrastructure to be built and deployed. Is it going to run on a server in your datacenter, on a virtual machine in the cloud, on a container, or on someone's laptop? Depending on your answer, you have another set of responsibilities related to infrastructure. Once you set up your infrastructure, you will need to answer a new series of questions, including:

- Are the coders developing this on the exact same platform on which it will be deployed?
- Who is setting up all the infrastructure?
- Does the infrastructure fail open or fail safe, in the case of an error?

That set of questions needs another set of tests to make sure that the application code is being run correctly on the right platform, and that the platform is not misbehaving.

In traditional software development, only *developers* take care of development, which means their prime directive is writing code to a specification. The *operations* team handles the environment and method of deployment, and the change management procedures. *Testers* take the output of the developers and the operations team and make sure the near-final product does not break. These three roles are not mutually exclusive, but each team needs to wait for input from the previous team to start its work.

A very common representation of this flow is the Software Development Life Cycle (SDLC) model (see [Figure 1-1](#)). In practice, the waterfall model of the SDLC might operate something like this: developers create code quickly, based on functional requirements. The code is sent for testing, errors are found, and the code is sent back to developers. The developers fix the code and send it for another round of testing. Once the testing is complete, the code is handed off to the operations team for maintenance.

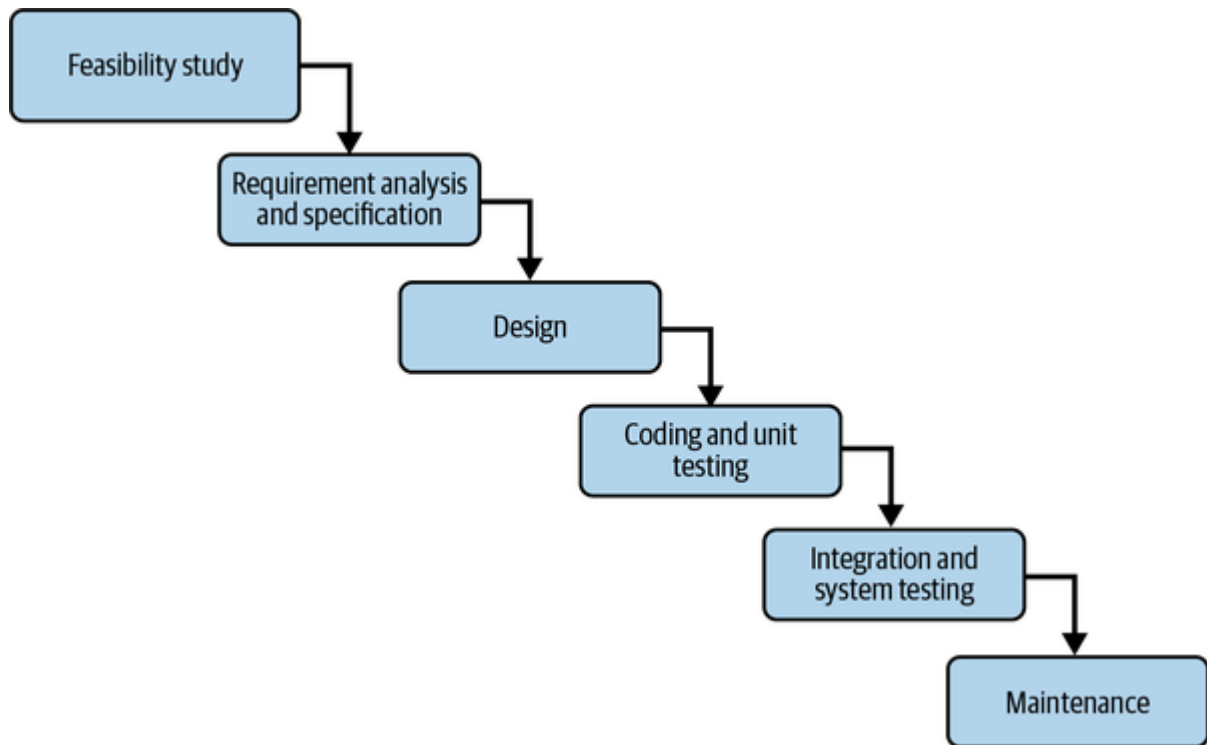


Figure 1-1. Software Development Life Cycle: waterfall model (<https://oreil.ly/KtVv0>)

Siloed teams operating in a hands-off style, similar to the SDLC in **Figure 1-1**, have their disadvantages. Each team has its own toolset and handles a very specific piece of the SDLC, and is typically unaware of the toolsets of the other teams. This makes it difficult to get quality software out the door on time.

The waterfall model leads to a lot of back-and-forth between teams, as we've noted. The back-and-forth is made worse when you have not delivered anything because your code has not passed your testing teams. So, a whole lot of time and effort is lost without producing any tangible outcomes.

Enter DevOps. In order to reduce time to market and improve the quality of software, the concept of DevOps was introduced. In their book *Effective DevOps* (O'Reilly), authors Jennifer Davis and Ryn Daniels define DevOps as:

A cultural movement that changes how individuals think about their work, values the diversity of work done, supports intentional processes that accelerate the rate by which businesses realize value, and measures the effect of social and technical change. It is a way of thinking and a way of working that enables individuals and organizations to develop and maintain sustainable work practices. It is a cultural framework for sharing stories and developing empathy, enabling people and teams to practice their crafts in effective and lasting ways.

In a DevOps model, the development, testing, and operations teams don't work in silos, but are the same group. If we were to visualize the DevOps model, it would look like a homogeneous blob of roles. Kief Morris writes in *Infrastructure as Code* that the goal of DevOps is:

To reduce barriers and friction between organizational silos—development, operations, and other stakeholders involved in planning, building, and running software. Although technology is the most visible, and in some ways simplest face of DevOps, it’s culture, people, and processes that have the most impact on flow and effectiveness. Technology and engineering practices like Infrastructure as Code should be used to support efforts to bridge gaps and improve collaboration.

We want to emphasize this point: *DevOps is not solely enabled by technology. It is effective only when people, processes, and technology are working together.* There is a common misconception that if you use tools that are used in CI/CD systems, you’re automatically practicing DevOps. This is flawed thinking. What enables DevOps is collaboration.

RECOMMENDATIONS FOR FURTHER READING

The basics of DevOps, modern architectures, and application security are all outside the scope of this book, but we recommend the following references if you want to learn more:

DevOps

- *Effective DevOps* by Jennifer Davis and Ryn Daniels (O’Reilly)
- *Understanding Agile DevOps* by Jim Bird (O’Reilly)
- *DevOps Tools for Java Developers* by Stephen Chin, Melissa McKay, Ixchel Ruiz, and Baruch Sadogursky (O’Reilly)

Modern architectures

- *Building Microservices, 2nd edition*, by Sam Newman (O’Reilly)
- *Building Event-Driven Microservices* by Adam Bellemare (O’Reilly)
- *Fundamentals of Software Architecture* by Mark Richards and Neal Ford (O’Reilly)

Application security

- *Shifting Left for Application Security* by Peter Conrad (O’Reilly)
- *Agile Application Security* by Laura Bell, Michael Brunton-Spall, Rich Smith, and Jim Bird (O’Reilly)

What Is DevSecOps?

DevSecOps is not a “new version” of DevOps, but rather a conscious effort to add security into your DevOps framework. Like with DevOps, there are numerous definitions of and approaches to DevSecOps. For the purposes of this book, we define *DevSecOps* as the ability to implement and deploy SaC in software.

We will be leaning heavily on APIs, cloud services, and other open source projects to implement SaC. When a part of your SDLC becomes “as code,” your team should have the openness to build things.

As your organization begins to implement DevSecOps, there are two important things your team will need to outline: tools and security guidelines.

First, how will you write your IaC? In other words, what tools will you use? This could be a tool like AWS CloudFormation or Terraform. There are numerous services and products available from vendors and the open source community that you can use to build and integrate SaC into your pipeline. As we mentioned, this book will use AWS and open source projects to demonstrate the *why* and *how* of doing DevSecOps, instead of fixating on a particular tool’s licensing or procurement. We chose to focus on AWS in this book since it is currently the most popular cloud infrastructure vendor, controlling 33% of the market. However, the book’s underlying principles will be useful to all readers.

Second, what are your company’s security “rules of the road”? What has your security team designated as “definitely don’t do this” rules? Understanding why the security team provides certain guidance helps you understand the concerns underlying the rules.

In DevSecOps, you are building security directly into your software development pipeline (see Figure 1-2). In step 1, the developer lints their code locally and makes sure its formatting follows the team’s conventions and standards. They then commit the code to the repository. In step 2, the build system of the pipeline scans for errors and any other vulnerabilities and misconfigurations. The security of the pipeline is built into this stage. Finally, in step 3, the code is (possibly) ready for deployment. The last hurdle is a *decision gate*—a mechanism that checks for errors. If any errors are found, deployment is canceled and the developer is informed. If the code has no errors, the deployment goes through.

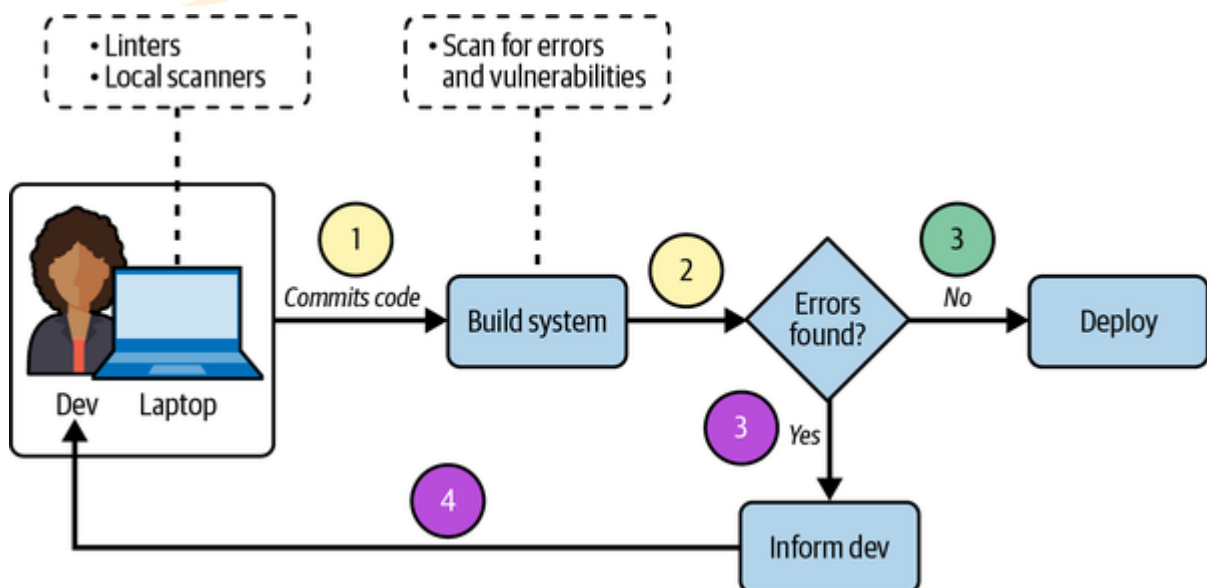


Figure 1-2. Building security into a DevOps pipeline

Steps 1 and 2 happen every time. The top version of step 3 only occurs when the linters and scanners discover no errors. Both the lower version of step 3 and step 4 only occur when the linters and scanners discover an issue.

The code that is deployed is not the only thing that needs to be secured. You also need to protect the security checks you are implementing within the DevOps pipeline. Imagine for a moment that anyone who commits code can turn security checks on and off. Would it be secure?

We have seen teams bypass their security checks because an “urgent” code change was needed and it was faster to deploy without security. That is a recipe for disaster. Situations where a real emergency justifies a nonsecured code change should be recorded and remediated. There should not be a bypass function for deployments.

Introducing Automatoonz

Security is a broad topic. Its subfields include physical security, application security, cryptography, training, and many more. We will limit our scope in this book to a subset of security domains, which we’ll explore by following the journey of an imaginary company we’ll call Automatoonz.

Automatoonz is an animation company that’s recently been getting into web toons and series creation. The company operates on AWS and is currently trying to build security into its development and deployment processes. It wants to be lean and secure, but the company is relatively new in the space and doesn’t have the funding to hire more security engineers. Thus, its aim is to automate as much security as possible.

Because Automatoonz operates on AWS, there are some security responsibilities it doesn’t have to worry about, such as physical security. AWS refers to this as the **shared responsibility model**.

Cloud Infrastructure: Secure by Default

In cloud infrastructure, every resource is created through an API call. Each API call has parameters that configure your resource exactly as you want it. The software development kit (SDK) you use will also automatically generate a default configuration, to provide maximum customization options.

For example, if you are using the AWS SDK for Python (Boto3) to create an Amazon S3 bucket, the only required parameter is a name for the bucket. The SDK does not ask you for encryption, nor does it require you to ensure the bucket is not publicly accessible. If you want to enforce encryption on the contents of the bucket, you should do so upon deployment. This is what is referred to as *Secure by Default*.

Secure by Default does not mean you should lock down everything you possibly can. It’s often said that the safest computer system is the one that’s unplugged. But, if you make your system so secure that it’s practically unusable—well, people won’t use it. They’ll look for workarounds, potentially compromising security in the process. In this book, we aim to demonstrate *usable security*—a balance between usability and security.

In an IaC paradigm, you can *templatize* your resources—make declarative code for patterns you come across and create templates that developers can work from, in order to keep the codebase consistent. For example, if you are building a web server within AWS, you should be able to standardize the architecture for future deployments. If the architecture is defined only in a diagram, you'll need to codify that diagram as IaC. To do so, you'll need to create declarative templates so that the same web server pattern can be deployed repeatedly with zero deviation.

What does templatzation provide in terms of security? Let's revisit the S3 bucket example. At Automatoonz, the legal team is tasked with ensuring that all of the company's data is *encrypted at rest*, meaning that data stored in any persistent storage has to be encrypted through a cryptographic key, which is only accessible to authorized users. In cloud computing, you can define encryption at rest through API flags in your IaC templates. This also helps standardize a Secure by Default posture for your infrastructure.

Of course, the legal team informs the developers of this requirement, and the developers need to apply the right checks and balances to adhere to the policy. If there is an IaC template for the bucket, the developers must check it before it is sent to AWS endpoints for resource creation. This will prevent the unencrypted bucket from ever being created.

At a very high level, **Figure 1-3** shows what we are trying to achieve. The developer on the left creates an AWS CloudFormation template, which in turn creates an S3 bucket. This goes to a decision gate. If encryption is not enabled, the decision gate shuts down deployment. Only after the template is fixed can the deployment go through.

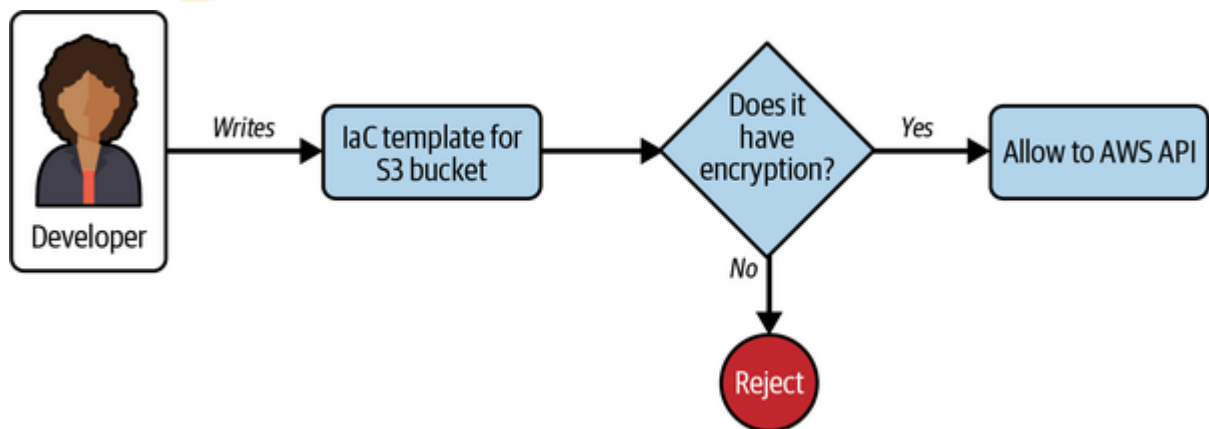


Figure 1-3. Decision gate that rejects unencrypted S3 bucket templates

Move Fast, Secure Fast: The Importance of Automation

When you drive a car, there are certain rules of the road that you must obey. Not adhering to these rules can lead to mishaps, serious accidents, and legal trouble. In security, we have similar invariable rules. Sometimes the rules are driven by regulatory or compliance requirements; other times they're intended to create engineering excellence. Regardless of the reason, every team is expected to follow these rules.

How can developers ensure they consistently follow rules related to security? Recall our earlier discussion of IaC, and its ability to define cloud resources in a declarative manner. Herein lies our answer. If we can set predictable flags and/or parameters for cloud resources, we can allow operations such as creating new resources or deploying new code to go through *only* if certain things are present.

When you want to move at the speed of the cloud, you want to secure things at the speed of the cloud as well. This means automating as much of your security as possible. There is a manual way of doing the check shown in **Figure 1-3**, in which the developer looks at their code and prevents a misconfiguration. But remember the old adage, “Humans are the weakest link in security”? You ideally want to automate these checks. To automate a security check—a central idea in DevSecOps—there are certain attributes to consider:

Idempotence

The security check should yield the same results no matter how many times you pass the template in question.

Baseline

The security check should apply to all or most resources. If a security check only applies to one variant of a resource, which appears in less than 30% of your deployments, you will get a lot of noise and failures while doing the check.

Recommendation

You found a security misconfiguration. That’s cool, but did you tell the build system what to do? If you leave your end users without a fix, you are adding to their work. Your scans should recommend fixes when a misconfiguration is found.

We will be using these attributes as we build our pipeline throughout the book. One thing we want to reiterate: automation does *not* solve everything. Without the right owners and processes, all the automation in the world will not amount to anything. These security checks need to be maintained, and the developer should give feedback to improve efficiency.

DevSecOps Culture

Technology is not a silver bullet. Culture plays a role, too. DevOps at its core is built on a culture, and its tools are only useful when people use them in the right ways.

We have seen organizations acquire every possible tool that claims to solve security issues, yet not even configure half of them. Why aren’t the teams using these security tools? Do they lack training? Are the tools overkill, or too expensive to roll out to the entire organization?

We highly recommend answering one simple question before you buy any security tool: *What security problem are you trying to solve?* Answers like “make things secure” and “improve my security” are as vague as it gets. Try to boil it down to a single sentence. If that doesn’t give you a clear answer, consider writing a risk statement. A risk statement looks something like this:

This document was truncated here because it was created in the Evaluation Mode.

